

MiniLedger

Core Rust, Decoded

*Every sign, symbol, and method of one small program,
reasoned out from first principles.*

by Subash Acharya

A guided study build — from the five roots up.

`rustc-verified / std-only / read top to bottom`

Preface

This is not a Rust reference. It is one very small program, read so closely that every sign, symbol, keyword, and method call in it becomes a doorway into core Rust. Learn this one file completely and you will have met, in context, most of the machinery you use day to day.

The program is **MiniLedger**: a toy fintech ledger that deposits, withdraws, and transfers money across worker threads into one shared balance map. It is deliberately overloaded. Every section was chosen to force a different language feature into view.

How this book reasons

Most tutorials explain Rust by analogy (“it is like a pointer, but...”) or by convention (“you write it this way”). We do the opposite. We start from the handful of things Rust is **forced** to be true about, and then show how each strange symbol falls out of them as a consequence rather than a choice. When a feature appears, we trace it back to one of five roots laid out in Chapter 0 and tag it like this: **(Root 2)**.

How to read it

- Keep the source open beside you. Appendix C is the full annotated file.
- Each chapter ends where a feature lives in the program, so you can jump to the line and see it working.
- The orange boxes are the first-principle insights. If you remember nothing else, remember those.
- Appendix A lists every symbol; Appendix B lists every method call. Use them as a lookup once you have read the chapters.

Contents

Preface	2
0 How Rust Thinks: The Five Roots	7
Root 1 — Memory is managed at compile time, not by a garbage collector	7
Root 2 — Aliasing XOR mutation	7
Root 3 — Abstractions are zero-cost by default	7
Root 4 — Errors and absence are ordinary values	7
Root 5 — Make illegal states unrepresentable	8
1 The Shape of a Program	9
Comments: <code>//</code> , <code>///</code> , and <code>#![</code>	9
use and the path separator <code>::</code>	9
fn, the entry point, and the <code>;</code> terminator	9
2 Values and Bindings	10
let, mut, and type annotations <code>(:)</code>	10
The primitive types in play	10
Owned String vs borrowed <code>&str</code> ;	10
Compound values: tuples <code>()</code> , arrays <code>[]</code> , slices <code>&[]</code>	11
Shadowing	11
const and static	11
Numeric literal sugar: <code>_</code> and suffixes	11
3 Ownership, Move, and Copy	12
The move	12

Copy: when a move is just a bitwise duplicate	12
.clone() and drop	12
4 References and Borrowing	13
& shared borrow, &mut; exclusive borrow, * dereference	13
Why &self; vs &mut; self on methods	13
Foreshadowing: this is why Mutex exists	13
5 Lifetimes	14
The 'a syntax on a struct that holds a borrow	14
'a on a function that returns a borrow	14
The anonymous lifetime ' _	14
6 Modeling Data: struct and enum	15
struct: three flavors	15
enum: alternatives that carry data	15
#[derive(...)] attributes	15
Default and the struct-update .. syntax	16
7 Traits, Part I: Shared Behavior	17
Declaring a trait, and the supertrait colon	17
self vs Self	17
Implementing a trait, and overriding a default	17
Display, Debug, and the write! macro	18
Operator overloading via traits: Add and its associated type	18
8 Traits, Part II: Generics and Dispatch	19

Associated types: type Item in a custom Iterator	19
Generic functions, trait bounds, and where	19
Static dispatch (monomorphization) vs dynamic dispatch (dyn)	19
impl Trait in return position	20
The turbofish ::<>	20
9 Errors and Absence as Values	21
Option: maybe a value	21
Result: success or a typed failure	21
A custom error type with Display, Error, and From	21
The ? operator	21
10 Closures and Iterators	23
Closures and the ... syntax	23
The Fn / FnMut / FnOnce family (this is Root 2 again)	23
move closures	23
Implementing Iterator yourself	23
Iterator adapters used in the program	24
11 Pattern Matching	25
match, the => arm, _, and destructuring	25
Ranges (.. and ..=), guards, and the @ binding	25
if let and while let	25
Binding modes: the exact error you hit, from first principles	26
12 Concurrency: Threads, Arc, Mutex, Channels	27
Spawning threads and joining them	27

Shared ownership with Arc, exclusive access with Mutex	27
Send and Sync: the invisible guarantees	28
Message passing with mpsc channels	28
13 Putting the Pointers Together	29
Appendix A Every Symbol in the Program	30
Appendix B Every Method and Macro Call	32
Appendix C The Complete Annotated Source	33

0 How Rust Thinks: The Five Roots

Before any syntax, hold these five truths. Nearly every symbol in MiniLedger is one of them wearing a costume.

Root 1 — Memory is managed at compile time, not by a garbage collector

C makes you free memory by hand (easy to get wrong). Java/Go run a garbage collector (a runtime cost and pauses). Rust takes a third path: each value has exactly one **owner**, and when the owner goes out of scope the value is freed automatically. No GC, no manual free. The compiler proves it is correct.

Consequence

Because ownership is singular, assigning or passing a value **moves** it by default. `move`, `.clone()`, `Copy`, `drop`, and even the word `Arc` all exist to manage who owns what and for how long.

Root 2 — Aliasing XOR mutation

A data race needs two things at once: shared access and mutation. Rust forbids the combination. At any moment a value may have **either** any number of shared references `&T` **or** exactly one exclusive reference `&mut T`, never both. This single rule, checked by the borrow checker, is why Rust is memory-safe and data-race-free.

Consequence

`&`, `&mut`, `*`, lifetimes (`'a`), `Mutex`, and the `Send/Sync` traits are all bookkeeping for this one rule. When the borrow checker fights you, it is enforcing Root 2.

Root 3 — Abstractions are zero-cost by default

A generic function is compiled into a separate specialized copy for each concrete type you use (**monomorphization**), so there is no runtime dispatch cost. You only pay for runtime flexibility when you explicitly ask for it with `dyn`.

Consequence

`<T>`, `impl Trait`, and the turbofish `::<>` are the cheap static path. `dyn` and `Box<dyn Trait>` are the opt-in dynamic path with a small vtable cost.

Root 4 — Errors and absence are ordinary values

There are no exceptions and no `null`. “Might be missing” is the type `Option<T>`. “Might fail” is the type `Result<T, E>`. Because these are normal enums, the compiler forces you to handle both arms.

Consequence

`Option`, `Result`, the `?` operator, and `From` (for error conversion) are how this root shows up. `?` is just early-return sugar over a `match`.

Root 5 — Make illegal states unrepresentable

If a value can only legally be one of a fixed set of shapes, model it as an `enum` whose variants carry exactly the data that shape needs. The type then makes wrong combinations impossible to construct, so you cannot forget a case.

Consequence

`enum` with data, `match` (which must be exhaustive), and the type system used as a modeling tool, all trace to Root 5.

1 The Shape of a Program

Open the top of the file. Before any logic there are comments, imports, and the symbols that hold a Rust program together.

Comments: `//`, `///`, and `//!`

```
//! MiniLedger - one tiny, overloaded tour of core Rust. (inner doc, whole file)
// acquire the Mutex (ordinary note)
/// the associated type the Iterator trait demands (doc for the item below)
```

- `//` is an ordinary comment ignored by the compiler.
- `///` is a **doc comment**: it documents the item written directly after it and is picked up by `cargo doc`.
- `//!` is an **inner doc comment**: it documents the thing it is inside (here, the whole file/module). Note the `!` meaning “the enclosing item”.

use and the path separator `::`

```
use std::sync::{mpsc, Arc, Mutex};
use std::collections::HashMap;
```

`::` is the **path separator**, the way you walk a namespace tree: `std` (the standard library) then `sync` (a module) then `Mutex` (a type). `use` simply brings a path into scope so you can write `Mutex` instead of the full `std::sync::Mutex` every time. The `{mpsc, Arc, Mutex}` braces import several items from the same parent path at once.

Why imports matter for traits

Recall from our earlier discussion: a trait’s methods are only callable where the trait is in scope. `use` is what puts it there. This is why adding a method sometimes means adding a `use`.

fn, the entry point, and the `;` terminator

```
fn main() -> Result<(), Box<dyn Error>> {
    // ...
    Ok(())
}
```

- `fn` declares a function. `main` is where execution begins.
- `->` introduces the **return type**. Here `main` returns a `Result`, which is what lets us use `?` inside it (Root 4; Chapter 9).
- `;` ends a **statement**. A line without a trailing `;` is an **expression** whose value becomes the block’s result. That is why the final `Ok(())` has no semicolon: it is the return value, not a statement.

Statements vs expressions (Root, quietly)

In Rust almost everything is an expression that produces a value: `if`, `match`, and blocks `{ }` all evaluate to something. The presence or absence of `;` decides whether you keep that value or throw it away.

2 Values and Bindings

This chapter covers the `let` block in `main` that declares one of every basic kind of value, plus `const` and `static` from the top of the file.

let, mut, and type annotations (:)

```
let immutable = 42;           // inferred i32, immutable by default
let mut counter: u32 = 0;     // explicit type u32, mutable
let pi: f64 = 3.14159;       // floating point
```

- `let` binds a name to a value. Bindings are **immutable by default** (a quiet nudge from Root 2: no accidental mutation).
- `mut` opts a binding into mutability. You only write it when you intend to change the value.
- The colon `:` after a name is a **type annotation**. `counter: u32` says “treat this as a `u32`.” When omitted, the compiler infers the type from how the value is used.

One colon, two jobs

You will see `:` again in `trait Pattern: Sized`. There it is **not** a type annotation; it declares a supertrait bound (Chapter 7). Same character, different grammar slot. Context tells them apart.

The primitive types in play

Binding	Type	What it is
<code>42</code>	<code>i32</code>	32-bit signed integer (the default integer type)
<code>counter</code>	<code>u32</code>	32-bit unsigned integer (never negative)
<code>pi</code>	<code>f64</code>	64-bit floating point
<code>grade = 'A'</code>	<code>char</code>	one Unicode scalar value, 4 bytes (not one byte)
<code>active = true</code>	<code>bool</code>	<code>true</code> or <code>false</code>
<code>cents</code>	<code>i64</code>	64-bit integer; we store money as whole cents

Owned String vs borrowed `&str`

```
let owned: String = String::from("syndication"); // owns its bytes, on the heap
let view: &str = &owned[0..4];                 // borrows a window -> "synd"
```

This distinction is pure Root 1. A `String` **owns** a growable buffer of UTF-8 bytes on the heap; when it drops, the buffer is freed. A `&str` is a **borrowed view** into bytes owned by something else: a pointer plus a length, owning nothing. `&owned[0..4]` takes a slice of the first four bytes. Because `view` borrows `owned`, Root 2 says `owned` cannot be moved or mutated while `view` is alive.

- Take `String` when you need ownership (store it, return it, mutate it).
- Take `&str` when you only need to read characters you do not own. Function parameters usually want `&str`.

Compound values: tuples `()`, arrays `[]`, slices `&[]`

```
let trio: (u8, &str, f64) = (1, "x", 0.5); // tuple: fixed, mixed types
let arr: [i64; 3]         = [10, 20, 30];  // array: fixed length, one type
let slice: &[i64]         = &arr[1..];     // slice: a borrowed view -> [20, 30]
```

- `()` build a **tuple**: a fixed-size, ordered group that may mix types. Access by position (`trio.0`) or by destructuring.
- `[T; N]` is an **array**: `N` values of one type `T`, length known at compile time.
- `&[T]` is a **slice**: a borrowed view into a run of array/vector elements (pointer + length). `arr[1..]` means “from index 1 to the end.” The `..` here is a range; more on ranges in Chapter 11.

Empty tuple `()` is the unit type

`()` with nothing inside is **unit**: the type with exactly one value, used to mean “no meaningful value.” A function that returns nothing returns `()`. That is why `Result<(), E>` reads as “succeeds with nothing, or fails with E.”

Shadowing

```
let shadowed = immutable;           // i32
let shadowed = shadowed.to_string(); // same name, now a String
```

Rust lets you reuse a name with a fresh `let`. This is **shadowing**, not mutation: the old binding still existed, you simply created a new one. It is the idiomatic way to transform a value through stages (and even change its type) without inventing new names.

const and static

```
const FEE_BPS: i64 = 50;           // a compile-time constant, inlined at use sites
static APP_NAME: &str = "MiniLedger"; // a single fixed value with a fixed address
```

- `const` is a value computed at compile time and effectively pasted in wherever used. It has no fixed memory location. Convention names it in `SCREAMING_SNAKE_CASE`.
- `static` is a single instance that lives for the entire program and has one fixed address. Use it for genuinely global data.
- Both **require** an explicit type annotation, because there is no surrounding code to infer from.

Numeric literal sugar: `_` and suffixes

```
100_000    // underscores are visual separators only; this is 100000
1_i64      // the suffix forces the literal's type to be i64
```

The underscore in `100_000` is ignored by the compiler and only helps you read large numbers. A type suffix like `1_i64` pins an otherwise-ambiguous literal to a specific type, which we needed in `fee_schedule` so the range produced `i64` values that could be multiplied by `FEE_BPS`.

3 Ownership, Move, and Copy

This is the beating heart (Root 1). Get it and the borrow checker stops feeling random.

The move

Every value has exactly one owner. Binding or passing a value transfers ownership; the old name becomes unusable. Consider what would happen with a `String`:

```
let a = String::from("hi");
let b = a;           // ownership MOVES from a to b
// println!("{a}"); // ERROR: a was moved, it no longer owns anything
```

This is not a performance trick, it is correctness. If both `a` and `b` could free the same heap buffer, you would get a double-free. The move makes that impossible by making `a` unusable. When `b` drops at the end of scope, the buffer is freed exactly once.

Copy: when a move is just a bitwise duplicate

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct Cents(pub i64);
...
let total = a + b; // a and b are STILL usable afterward, because Cents is Copy
```

Small, plain types whose bytes mean the same thing wherever they are copied (integers, `char`, `bool`, and structs of such things that opt in with `#[derive(Copy)]`) implement the `Copy` trait. For these, assignment **copies** instead of moving, so the original stays valid. `Cents` wraps a single `i64`, so we let it be `Copy`: the math `a + b` does not consume `a` and `b`.

Move vs Copy is decided by the type

If a type owns a heap resource (like `String` or `Vec`), it is **not** `Copy`; duplicating it must be an explicit `.clone()` so the cost is visible. If a type is just plain bits, it can be `Copy` and duplication is free and silent.

`.clone()` and `drop`

```
account.clone() // make an independent owned copy (here, of a String key)
drop(tx);       // end this value's life right now, before scope end
```

- `.clone()` produces a brand-new, independently owned value. We call it on `account` because inserting into the map needs an owned `String` key while we still want to keep using the original.
- `drop(x)` ends a value's life immediately. We use it on the channel sender so the channel can close (Chapter 12). Normally values drop automatically at the end of their scope; `drop` just does it sooner.

4 References and Borrowing

If moving were the only option you could never look at a value without consuming it. Borrowing is the fix, and it is governed entirely by Root 2.

& shared borrow, &mut exclusive borrow, * dereference

```
fn balance(&self, who: &str) -> i64 { ... }      // reads through a shared borrow
fn apply(&mut self, txn: &Txn) -> Result<...> { ... } // mutates through an exclusive borrow
*self.balances.get_mut(account).unwrap() -= cents;    // * to write through the reference
```

- `&x` creates a **shared reference**: read-only, and you may have many at once.
- `&mut x` creates an **exclusive reference**: read-write, and the compiler guarantees it is the **only** live reference to that value while it exists.
- `*r` **dereferences**: it follows the reference to the value it points at, so you can read or assign the value itself. `*ptr -= cents` subtracts from the pointed-to integer, not from the pointer.

This is Root 2, made of three symbols

Many `&` or one `&mut`, never both. `&self` methods can run concurrently with other reads; a `&mut self` method needs the whole value to itself. The entire borrow checker is the enforcement of that one sentence.

Why `&self` vs `&mut self` on methods

A method's first parameter says how it borrows the receiver. `balance(&self, ...)` only reads, so it takes a shared borrow and any number of callers can read at once. `apply(&mut self, ...)` changes balances, so it demands exclusive access. The signature is a promise the compiler checks and the caller can rely on.

Foreshadowing: this is why Mutex exists

Across threads the compiler cannot prove who borrows what, so it will not let you share a `&mut` freely. A `Mutex` restores the guarantee at runtime: locking it hands you exclusive access for as long as you hold the lock, satisfying Root 2 dynamically. Chapter 12 returns to this.

5 Lifetimes

A reference must never outlive the value it points to, or it would be a dangling pointer. Lifetimes are the **names** the compiler uses to track how long borrows are valid. They describe relationships; they do not change how long anything actually lives.

The 'a syntax on a struct that holds a borrow

```
struct Report<'a> {
    title: &'a str,      // borrowed: tied to lifetime 'a
    lines: Vec<String>, // owned
}
```

`Report` stores a borrowed `&str`. Rust must know that the `Report` cannot outlive the string it borrowed. The lifetime parameter `'a` is how we say so: “this `Report` is valid only for some region `'a`, and `title` borrows for that same region.” The apostrophe-name (`'a`) is just a label; you could call it `'report`. Any struct that holds a reference **must** carry such a parameter. The `lines: Vec<String>` field owns its data, so it needs no lifetime.

'a on a function that returns a borrow

```
fn richest<'a>(&'a self) -> Option<(&'a str, i64)> {
    self.balances.iter()
        .max_by_key(|&(_, &bal)| bal)
        .map(|(name, &bal)| (name.as_str(), bal))
}
```

`richest` returns a `&str` that points **into** `self`’s map. The `'a` ties the returned reference to `self`: the caller may use the returned name only as long as the `Ledger` is alive. Without that link the compiler could not stop you from using the name after the ledger was freed.

Elision: the lifetimes you do not see

We wrote `'a` here to make it visible, but Rust would have inferred it. For one input reference and a borrowed output, the rule “output borrows from the single input” is automatic, so `fn richest(&self) -> Option<(&str, i64)>` compiles identically. Lifetimes are present on nearly every borrow; elision just hides the obvious ones.

The anonymous lifetime ' _

```
fn fmt(&self, f: &mut fmt::Formatter<' _>) -> fmt::Result
```

`' _` means “there is a lifetime here, infer it, I do not need to name it.” It documents that `Formatter` carries a borrow without forcing you to invent a label. You will see `' _` constantly in signatures copied from the standard library.

6 Modeling Data: struct and enum

Root 5 says: shape your types so wrong states cannot exist. Structs group data that belongs together; enums express “one of these alternatives.”

struct: three flavors

```
struct Ledger { balances: HashMap<String, i64> } // named-field struct
pub struct Cents(pub i64);                      // tuple struct (positional fields)
struct Ach;                                     // unit struct (no fields)
```

- A **named-field** struct labels each field. Access with `ledger.balances`.
- A **tuple struct** has positional fields. `Cents(pub i64)` wraps one `i64`; access it with `.0`. This is the **newtype** pattern: a distinct type that is just an `i64` underneath, so you cannot accidentally mix raw cents with, say, raw counts.
- A **unit struct** has no data. `Ach` exists purely to implement a trait on something (Chapter 7); it carries no state.

enum: alternatives that carry data

```
enum Txn {
    Deposit { account: String, cents: i64 },
    Withdraw { account: String, cents: i64 },
    Transfer { from: String, to: String, cents: i64 },
}
```

This is Root 5 in its purest form. A transaction **is** exactly one of three shapes, and each shape carries precisely the fields it needs. A `Transfer` has `from` and `to`; a `Deposit` does not. You cannot build a `Deposit` with a stray `from` field, because the type does not allow it. When you later `match` on a `Txn`, the compiler checks you handled every variant.

#[derive(...)] attributes

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct Cents(pub i64);
```

`#[...]` is an **attribute**: metadata attached to the item below it. `derive` asks the compiler to auto-generate standard trait implementations so you do not hand-write them:

Derive	What you get
<code>Debug</code>	<code>{:?}</code> formatting for printing/inspection
<code>Clone</code>	an explicit <code>.clone()</code> that deep-copies
<code>Copy</code>	implicit bitwise copy on assignment (Chapter 3)
<code>PartialEq / Eq</code>	the <code>==</code> operator
<code>Default</code>	a <code>::default()</code> value (here, <code>Cents(0)</code>)

Default and the struct-update `..` syntax

```
fn new() -> Self {  
    Ledger { ..Default::default() } // fill remaining fields from Default  
}
```

`Default::default()` produces the type's default value. Inside a struct literal, `..expr` means “take any fields I did not list from `expr`.” Here we list none, so every field comes from the default (an empty `HashMap`). `Self` (capital S) is shorthand for “the type this `impl` block is for,” here `Ledger`.

7 Traits, Part I: Shared Behavior

A trait is a named set of behavior that many types can implement. It is Rust's interface mechanism, and the foundation of both generics (Root 3) and error handling (Root 4).

Declaring a trait, and the supertrait colon

```
trait Rail: fmt::Debug {
    fn name(&self) -> &str;           // required method (no body)
    fn settle(&self, c: Cents) -> String { // default method (has a body)
        format!("[{}] settled {c}", self.name())
    }
}
```

- `trait Rail: fmt::Debug` declares a trait. The `: fmt::Debug` part is a **supertrait** bound, the same grammar as `trait Pattern: Sized` we discussed before. It means “to implement `Rail`, a type must also implement `Debug`.” It is a requirement on implementors, not a kind of inheritance.
- `fn name(&self) -> &str;` ends in `;` with **no body**: a **required method**. Every implementor must supply it.
- `fn settle(...) { ... }` has a body: a **default method**. Implementors get it for free and may override it. `Wire` overrides it; `Ach` does not.

Required + default = the trait's deal

You implement the few required methods, and the trait hands you the rest built on top. This is exactly how `Iterator` works (Chapter 10): you write `next`, you receive dozens of adapters.

self vs Self

- lowercase `self` is the **receiver value** (the instance the method is called on), borrowed as `&self`, `&mut self`, or taken by value as `self`.
- Capital `Self` is the **type** currently implementing the trait. In `Rail`, `Self::NAME` or a `-> Self` would refer to that concrete type.

Implementing a trait, and overriding a default

```
impl Rail for Ach {
    fn name(&self) -> &str { "ACH" } // only the required method; settle is inherited
}
impl Rail for Wire {
    fn name(&self) -> &str { "WIRE" }
    fn settle(&self, c: Cents) -> String { // override the default
        let tag = if self.priority { "PRIORITY" } else { "std" };
        format!("[WIRE/{tag}] settled {c}")
    }
}
```

`impl Trait for Type { ... }` provides the behavior. `Ach` supplies only `name`, so it uses the default `settle`. `Wire` supplies both, replacing the default. Note `if` is an expression here: the whole `if/else` evaluates to the

chosen `&str`, which binds to `tag`.

Display, Debug, and the write! macro

```
impl fmt::Display for Cents {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "${}.{:02}", self.0 / 100, (self.0 % 100).abs())
    }
}
```

- Implementing `Display` defines how `{}` prints your type. `Debug` (which we derived) defines `{:?}` for developer-facing output.
- `write!(f, ...)` writes formatted text into the formatter `f`. It is the same formatting engine as `println!`, just aimed at a buffer instead of the screen.
- `{:02}` is a **format spec**: pad with zeros to width 2. So `Cents(150)` prints as `$1.50`. The `{}` and `{:?}` you see everywhere are this same mini-language.

Operator overloading via traits: Add and its associated type

```
impl Add for Cents {
    type Output = Cents; // the result type of the + operator
    fn add(self, rhs: Cents) -> Cents { Cents(self.0 + rhs.0) }
}
```

Operators in Rust are just traits. Writing `a + b` calls `a.add(b)` through the `Add` trait. To make `Cents + Cents` work we implement `Add`. `type Output = Cents` is an **associated type**: it declares what `+` produces. We meet associated types again in the next chapter, where they are central to `Iterator`. Because `Cents` is `Copy`, `add(self, rhs)` consuming both operands is harmless: the originals are copied, not moved.

8 Traits, Part II: Generics and Dispatch

Here Root 3 (zero-cost abstraction) becomes concrete, and we settle the static-vs-dynamic question that motivated `match_indices` earlier.

Associated types: type `Item` in a custom `Iterator`

```
impl Iterator for Countdown {
    type Item = u32; // each implementor picks the element type
    fn next(&mut self) -> Option<u32> { ... }
}
```

An **associated type** is a type slot the trait declares and each implementor fills in. `Iterator` declares `type Item`; our `Countdown` sets it to `u32`. This is different from a generic parameter: a type implements `Iterator` **once**, with one chosen `Item`, rather than for many types at once. The `type Searcher<'a>` you saw inside `Pattern` is the same idea, with an added lifetime.

Generic functions, trait bounds, and where

```
fn describe_all<R>(rails: &[R]) -> Vec<String>
where
    R: Rail,
{
    rails.iter().map(|r| r.settle(Cents(100))).collect()
}
```

- `<R>` introduces a **generic type parameter**: this function works for any type `R`.
- `where R: Rail` is a **bound**: `R` is not just anything, it must implement `Rail`, which is what lets us call `r.settle(...)`. The `where` clause is just a roomier place to write bounds; `fn describe_all<R: Rail>(...)` means the same thing.
- Because `R` is named and used in the slice type `&[R]`, it must be a named parameter, not `impl Rail`. This is exactly the rule from our `match_indices` discussion: a type used more than once in a signature needs a name.

Static dispatch (monomorphization) vs dynamic dispatch (dyn)

```
// static: compiler generates a specialized copy per concrete R. Zero runtime cost.
describe_all(&[Ach, Ach]);

// dynamic: one code path, the concrete type chosen at runtime via a vtable.
let rails: Vec<Box<dyn Rail>> = vec![Box::new(Ach), Box::new(Wire { priority: true })];
for r in &rails { println!("{}", r.settle(Cents(999))); }
```

This is the whole tradeoff of Root 3. With generics, the compiler stamps out a separate, fully optimized version of `describe_all` for each concrete `R`: no runtime indirection, larger binary. With `dyn Rail`, you get **one** version that looks up the right `settle` through a hidden table of function pointers (a **vtable**) at run time: smaller, flexible, and able to hold a mix of types in one `Vec`, at the cost of a pointer hop. You reach for `dyn` exactly when you need a heterogeneous collection or to decide types at runtime.

Why dyn needs a pointer (Box)

A `dyn Rail` has no known size (an `Ach` and a `Wire` differ in size), so it cannot sit on the stack directly. `Box<dyn Rail>` puts it on the heap and holds a fixed-size pointer. This is the same unsized-ness that made `dyn Pattern` impossible earlier, here solved by boxing.

impl Trait in return position

```
fn fee_schedule() -> impl Iterator<Item = i64> {
    (1_i64..=3).map(|tier| tier * FEE_BPS)
}
```

`impl Iterator<Item = i64>` means “I return some concrete type that implements this trait, but I will not name it.” The real type of a `.map(...)` iterator is an unspeakable compiler-generated name, so `impl Trait` lets you return it without writing it. It is still static dispatch: one concrete hidden type, no vtable.

The turbofish ::<>

```
fee_schedule().collect::

```

`collect` can build many container types, so sometimes inference cannot tell which one you want. The **turbofish** `::<Vec<_>>` pins it: “collect into a `Vec`, infer the element type (`_`).” The `_` is “figure this out for me.”

9 Errors and Absence as Values

Root 4 in full. No exceptions, no null. Two enums carry the weight.

Option<T>: maybe a value

```
self.balances.get(who).copied().unwrap_or(0)
```

`HashMap::get` returns `Option<i64>: Some(&n)` if the key exists, `None` if not. `Option` forces you to deal with absence. `.copied()` turns `Option<i64>` into `Option<i64>` (copying the integer out), and `.unwrap_or(0)` says “use the value, or 0 if `None`.” No null, no surprise.

Result<T, E>: success or a typed failure

```
fn apply(&mut self, txn: &Txn) -> Result<(), LedgerError> { ... Ok(()) }
```

`Result` has two variants: `Ok(value)` and `Err(error)`. `apply` returns `Result<(), LedgerError>`: on success it carries unit `()` (nothing useful, the work was a side effect), on failure a `LedgerError`. The caller cannot ignore the error arm without the compiler noticing.

A custom error type with Display, Error, and From

```
enum LedgerError {
    InsufficientFunds { account: String, needed: i64, have: i64 },
    BadAmount(std::num::ParseIntError),
}
impl Error for LedgerError {} // opt into the std error trait
impl From<std::num::ParseIntError> for LedgerError { // enables auto-conversion
    fn from(e: std::num::ParseIntError) -> Self { LedgerError::BadAmount(e) }
}
```

- The error is itself an `enum` (Root 5): a failure is exactly one named reason carrying exactly the data that reason needs.
- `impl Error for LedgerError {}` opts the type into the standard `Error` trait (an empty body is fine because `Error` has sensible defaults and requires only `Debug` + `Display`, which we provide).
- `impl From<ParseIntError>` teaches the compiler how to turn a parse error into our error. This is the hook the `?` operator uses, next.

The ? operator

```
fn parse_amount(s: &str) -> Result<i64, LedgerError> {
    let cents: i64 = s.trim().parse()?; // on Err, convert via From and return early
    Ok(cents)
}
```

`?` is the centerpiece of Root 4. Placed after a `Result`, it means: if `Ok(v)`, unwrap to `v` and continue; if `Err(e)`, **return early** from the function with that error, first converting it through `From` if the error types differ. Here `parse()` yields a `ParseIntError`, but our function returns `LedgerError`; the `?` silently calls the `From` impl above. Expanded, `x?` is roughly:

```
match x {
```

```
    Ok(v) => v,  
    Err(e) => return Err(LedgerError::from(e)),  
}
```

? in main

`main` returns `Result<(), Box<dyn Error>>` precisely so we can use `?` at the top level. `Box<dyn Error>` is a trait object (Chapter 8) that can hold any error type, so every concrete error converts into it. That is why a `LedgerError` from `parse_amount` can `?`-propagate straight out of `main`.

10 Closures and Iterators

Two of Rust's most-used tools, both leaning on traits and Root 3.

Closures and the `|...|` syntax

```
let with_fee = apply_fee(10_000, |x| x + x * FEE_BPS / 10_000); // Fn
let mut bump = || counter += 1;                               // FnMut
```

A **closure** is an anonymous function that can capture variables from its surroundings. The `|params|` bars hold its parameters (empty bars `||` mean no parameters). `|x| x + ...` takes one argument; `|| counter += 1` takes none but captures `counter`.

The Fn / FnMut / FnOnce family (this is Root 2 again)

Trait	How it captures	Example here
Fn	borrowes captures by <code>&</code> (read-only); callable many times	<code> x x + x * FEE_BPS / 10_000</code>
FnMut	borrowes by <code>&mut</code> (mutates captures); callable many times	<code> counter += 1</code>
FnOnce	takes captures by value (consumes them); callable once	<code>move</code> closures in threads

Which trait a closure implements is decided automatically by what it does to its captures, and it is the same aliasing-XOR-mutation rule (Root 2) in yet another costume. `bump` mutates `counter`, so it is `FnMut` and must itself be `mut`. `apply_fee<F: Fn(i64) -> i64>` accepts any closure that only reads.

move closures

```
thread::spawn(move || { ... use ledger, tx, batch ... })
```

`move` forces the closure to **take ownership** of everything it captures rather than borrowing. A thread may outlive the function that spawned it, so borrowing local variables would risk a dangling reference (Root 1 + Root 2). `move` hands the captured values to the thread outright, sidestepping the problem. Chapter 12 uses this.

Implementing Iterator yourself

```
struct Countdown { n: u32 }
impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.n == 0 { None }
        else { self.n -= 1; Some(self.n + 1) }
    }
}
```

An iterator is any type that can produce a `next` item. The contract is tiny: one associated type `Item` and one required method `next`, which returns `Some(item)` until it returns `None` to signal the end. `Countdown { n: 3 }` yields `3`, `2`, `1`, then `None`. Because you satisfied that contract, the trait hands you every adapter below for free, the required-plus-default pattern from Chapter 7 at full power.

Iterator adapters used in the program

Call	What it does
<code>.iter()</code>	borrow each element (<code>&T</code>); does not consume the collection
<code>.into_iter()</code>	consume the collection, yielding owned items
<code>.enumerate()</code>	pair each item with its index: <code>(0, a), (1, b), ...</code>
<code>.map(f)</code>	transform each item with closure <code>f</code>
<code>.max_by_key(f)</code>	the item whose <code>f(item)</code> key is largest
<code>.collect()</code>	gather items into a collection (often with a turbofish)
<code>.sum()</code>	add the items together

Adapters like `map` are **lazy**: they describe work without doing it. A consuming call such as `collect`, `sum`, or a `for` loop is what actually pulls items through. That is why `fee_schedule()` can return an unconsumed `map` iterator and the caller decides when to run it with `.collect::<Vec<_>>()`.

11 Pattern Matching

Pattern matching is how Rust takes things apart. It pairs naturally with enums (Root 5): because a value is exactly one shape, matching lets you handle each shape with the right data already unpacked.

match, the `=>` arm, `_`, and destructuring

```
match txn {
    Txn::Deposit { account, cents } => { ... } // destructure the variant
    Txn::Withdraw { account, cents } => { ... }
    Txn::Transfer { from, to, cents } => { ... }
}
```

- `match value { pattern => result, ... }` compares `value` against each pattern top to bottom and runs the first arm that fits.
- `=>` separates a pattern from the code it triggers.
- `Txn::Deposit { account, cents }` is a **destructuring pattern**: it both matches the `Deposit` variant and binds its fields to the names `account` and `cents` for use in the arm.
- `match` must be **exhaustive**: every possible variant must be covered, or the program will not compile. This is Root 5 paying off; you cannot forget a case.

Ranges (`..` and `..=`), guards, and the `@` binding

```
let tier = match bal {
    0                => "empty".to_string(),
    n @ 1..=49_999   => format!("small({n})"), // @ binds the matched value to n
    50_000..=99_999 => "mid".to_string(),      // inclusive range pattern
    _                => "whale".to_string(),   // _ matches anything (catch-all)
};
```

- `1..=49_999` is an **inclusive range pattern**: matches any value from 1 through 49999. (`a..b` is exclusive of `b`; `a..=b` includes `b`.)
- `n @ pattern` is an **@ binding**: match the pattern **and** capture the whole matched value in `n`, so the arm can use the actual number.
- `_` is the **wildcard**: matches anything and binds nothing. As the last arm it is the catch-all that makes the `match` exhaustive.
- `match` is an expression: the chosen arm's value becomes `tier`.

if let and while let

```
if let Some((name, bal)) = final_ledger.richest() { ... } // run only if Some
while let Ok(msg) = rx.recv() { receipts.push(msg); }     // loop while pattern fits
```

- `if let pattern = expr { ... }` is a one-armed match: do the block only when the pattern matches, ignore everything else. Here, act only if `richest()` returned `Some`.
- `while let pattern = expr { ... }` loops as long as the pattern keeps matching. We drain the channel while `recv()` keeps returning `Ok`; when the channel closes it returns `Err`, the pattern fails, and the loop ends.

Binding modes: the exact error you hit, from first principles

Recall the compiler error on `max_by_key(|(_, &bal)| bal)`. This is worth dwelling on because it reveals a deep rule. `max_by_key` hands its closure a **reference** to each item, so the closure actually receives `&(&String, &i64)`.

```
.max_by_key(|(_, &bal)| bal)    // edition-2024 ERROR  
.max_by_key(|&(_, &bal)| bal)  // fixed: consume the outer & explicitly
```

When you match a non-reference pattern like `(_, &bal)` against a reference type, Rust turns on **default binding modes** (“match ergonomics”): it quietly borrows as it descends, so you can write `(a, b)` instead of `&(a, b)`. The catch in edition 2024: once you are in that implicitly-borrowing mode, you may **not** also write an explicit `&` deref pattern (`&bal`) inside it. The two ways of peeling a reference, implicit and explicit, are not allowed to mix. The fix is to take the outer reference yourself with a leading `&`; that resets the inner patterns to ordinary by-value mode, where `&bal` is allowed again.

The first principle underneath

A pattern must describe the shape of the value being matched. Match ergonomics was added as convenience but made the shape ambiguous when mixed with explicit `&`. Edition 2024 removed the ambiguity by forbidding the mix. Either be fully implicit `(_, bal)`, or be fully explicit `&(_, &bal)`. An edition-proof alternative avoids patterns entirely: `.max_by_key(|e| *e.1)`.

12 Concurrency: Threads, Arc, Mutex, Channels

Everything so far converges here. Concurrency is where Root 2 (aliasing XOR mutation) stops being theoretical: the compiler will not let you share mutable state between threads unless you prove it is safe.

Spawning threads and joining them

```
let handle = thread::spawn(move || { ... });
...
for h in handles { h.join().unwrap(); }
```

- `thread::spawn(closure)` starts a new OS thread running `closure` and returns a `JoinHandle`.
- `.join()` blocks until that thread finishes; it returns a `Result` (the thread might have panicked), so we `.unwrap()` to surface a panic.
- The closure is `move` because the thread may outlive `main`'s current scope, so it must **own** what it uses (Chapter 10).

Shared ownership with Arc, exclusive access with Mutex

```
let ledger = Arc::new(Mutex::new(Ledger::new()));
...
let ledger = Arc::clone(&ledger);           // each thread gets its own handle
let mut guard = ledger.lock().unwrap();      // acquire exclusive access
guard.apply(txn)?;                          // mutate safely
// guard drops at end of block -> lock released
```

Two problems, two tools, both from Root 1 and Root 2:

- **Who owns the ledger?** All threads need it, but Root 1 allows only one owner. `Arc<T>` (Atomically Reference-Counted) is a shared-ownership pointer: every `Arc::clone` bumps a counter, and the value is freed only when the last `Arc` drops. “Atomically” means the count is safe to update from multiple threads.
- **How do they mutate it without racing?** Root 2 forbids shared `&mut`. `Mutex<T>` restores the guarantee at runtime: `.lock()` blocks until no one else holds the lock, then returns a guard that gives `&mut` access. While you hold the guard you are the only writer. When the guard drops (end of the block) the lock releases.

`Arc<Mutex<T>>` is the canonical shared-mutable pattern

`Arc` answers “who owns it” (many, shared). `Mutex` answers “who may mutate it right now” (one, briefly). Together they let multiple threads safely write to one `Ledger`. The `.unwrap()` on `lock()` handles the rare case where another thread panicked while holding the lock (“poisoning”).

Send and Sync: the invisible guarantees

You never type them here, but two marker traits make all of this sound. **Send** means a value can be moved to another thread; **Sync** means it can be shared between threads by reference. The compiler auto-derives them and **refuses** to compile `thread::spawn` unless every captured value qualifies. `Arc<Mutex<Ledger>>` is **Send** + **Sync**; a plain `Rc` would not be, and the code would not compile. This is Root 2 enforced at the type level across thread boundaries.

Message passing with mpsc channels

```
let (tx, rx) = mpsc::channel:::<String>();
let tx = tx.clone();           // one sender per worker
tx.send(format!("worker {i}: ok")); // workers send results
drop(tx);                     // drop the original so the channel can close
while let Ok(msg) = rx.recv() { receipts.push(msg); } // main collects
```

Besides sharing memory, threads can pass **messages**. `mpsc` is “multi-producer, single-consumer.” `channel()` returns a sender `tx` and receiver `rx`. Each worker gets a clone of `tx`; the receiver stays in `main`. A worker calls `tx.send(value)`; `main` calls `rx.recv()` to pull values out. The channel reports “closed” (an `Err` from `recv`) only when **all** senders are dropped, which is why we `drop(tx)` on the original after cloning: otherwise the `while let` would wait forever for a sender that never sends.

Two styles, one choice

Shared state (`Arc<Mutex<_>>`) and message passing (`mpsc`) are the two ways threads coordinate. Share memory when many workers update one structure; pass messages when you want to hand results back without shared locks. MiniLedger uses both at once.

13 Putting the Pointers Together

Rust has several pointer-like types. They look similar but answer different questions, all rooted in ownership (Root 1) and borrowing (Root 2). Here is the whole family as it appears in MiniLedger.

Type	Owns?	Question it answers
<code>&T</code>	no	"Let me read this without taking it" (shared borrow)
<code>&mut T</code>	no	"Let me change this; I am the only one right now"
<code>Box<T></code>	yes	"Put this one value on the heap" (needed for <code>dyn</code> , recursion)
<code>Arc<T></code>	yes (shared)	"Many owners across threads; free when the last leaves"
<code>Mutex<T></code>	wraps	"Grant exclusive mutation at runtime" (not a pointer, but pairs with <code>Arc</code>)

A useful mental ladder: start with a plain value. Need to borrow it? `&` / `&mut`. Need it on the heap or behind a trait object? `Box`. Need shared ownership? `Rc` (single thread) or `Arc` (across threads). Need shared **mutable** ownership across threads? `Arc<Mutex<T>>`. Each step up the ladder is something the previous one could not express, never decoration.

Appendix A Every Symbol in the Program

A lookup table for each sign and sigil that appears in MiniLedger. “Root” points to the first principle it serves.

Symbol	Meaning	Root / Chapter
// /// //!	comment / doc comment / inner doc comment	Ch.1
;	statement terminator (its absence makes an expression a return value)	Ch.1
::	path separator (and ::<> turbofish)	Ch.1, 8
:	type annotation, or a trait/supertrait bound	Ch.2, 7
{ }	code block / struct literal / format placeholder	Ch.1, 6, 7
()	tuple, function call, and unit type ()	Ch.2
[]	array type [T; N], slice indexing, ranges	Ch.2
< >	generic parameters and arguments	Ch.8
&	shared reference / borrow	Root 2, Ch.4
&mut	exclusive (mutable) reference	Root 2, Ch.4
*	dereference (follow a reference to its value)	Root 2, Ch.4
'a '_	named lifetime / anonymous (inferred) lifetime	Root 2, Ch.5
_	wildcard pattern, or “infer this” in types	Ch.8, 11
?	error-propagation operator (early return + From)	Root 4, Ch.9
!	macro invocation (println!), and “enclosing item” in //!	Ch.1, 12
...	closure parameter list	Ch.10
->	function return type	Ch.1
=>	match arm separator	Ch.11
...=	range (exclusive / inclusive); also struct-update ..	Ch.2, 6, 11
@	bind the matched value while also matching a pattern	Ch.11
#[...]	attribute (e.g. #[derive(...)], #[cfg(test)])	Ch.6
=	assignment / binding	Ch.2
==	equality (from PartialEq)	Ch.6
+= -=	compound assignment	Ch.3, 4
+ * / %	arithmetic (and + is overloadable via Add)	Ch.7
.	field access and method call	Ch.3+
"..." 'A'	string literal (&str) / char literal	Ch.2
100_000 1_i64	digit separator / literal type suffix	Ch.2
{:?}", {:02}	Debug format / zero-padded width spec	Ch.7
Self self	the implementing type / the receiver value	Ch.6, 7

Appendix B Every Method and Macro Call

Each call MiniLedger makes into the standard library, and why.

Call	What it does / why it is here
<code>String::from, .into()</code>	build an owned <code>String</code> from a <code>&str</code>
<code>.clone()</code>	make an independent owned copy (needed for owned map keys)
<code>.to_string()</code>	render a value to a <code>String</code> (via <code>Display</code>)
<code>.as_str()</code>	borrow a <code>String</code> as a <code>&str</code>
<code>.trim()</code>	drop surrounding whitespace before parsing
<code>.parse()</code>	parse a <code>&str</code> into a number; returns <code>Result</code>
<code>HashMap::get</code>	look up a key; returns <code>Option<&V></code>
<code>.copied()</code>	turn <code>Option<&T></code> into <code>Option<T></code> for a <code>Copy T</code>
<code>.unwrap_or(d)</code>	the value, or default <code>d</code> if <code>None</code>
<code>.entry(k).or_insert(v)</code>	get a mutable slot for key <code>k</code> , inserting <code>v</code> if absent
<code>.get_mut(k)</code>	a mutable reference to a value by key
<code>.unwrap()</code>	extract <code>Ok/Some</code> , or panic; used where failure is a bug
<code>.iter() / .into_iter()</code>	iterate by reference / by value
<code>.enumerate()</code>	attach a running index to each item
<code>.map(f) / .collect() / .sum()</code>	transform / gather / total an iterator
<code>.max_by_key(f)</code>	pick the item with the largest key <code>f(item)</code>
<code>.push(x) / .sort()</code>	append to a <code>Vec</code> / sort it in place
<code>Arc::new / Arc::clone</code>	create / share a reference-counted owner
<code>Mutex::new / .lock()</code>	create a mutex / acquire exclusive access
<code>mpsc::channel</code>	create a sender/receiver pair
<code>.send() / .recv()</code>	push a message / pull one (blocking)
<code>thread::spawn / .join()</code>	start a thread / wait for it
<code>Box::new</code>	heap-allocate a value (here, behind <code>dyn Rail</code>)
<code>drop(x)</code>	end a value's life immediately (close the channel)
<code>Default::default()</code>	produce the type's default value
<code>println! format! write!</code>	print / build a <code>String</code> / write into a formatter
<code>vec![...] assert_eq!</code>	build a <code>Vec</code> / assert equality in the test

Appendix C The Complete Annotated Source

The whole program, verified to compile and run. Read it now that every symbol has a name.

```

//! MiniLedger – one tiny, deliberately overloaded tour of core Rust.
//! Std-only, single file. Read top to bottom; every section is labeled.
//! Run it:          rustc miniledger.rs && ./miniledger
//! Run the test:    rustc --test miniledger.rs && ./miniledger

use std::collections::HashMap;
use std::error::Error;
use std::fmt;
use std::sync::{mpsc, Arc, Mutex};
use std::thread;

// === 1. const & static (compile-time values) ===
const FEE_BPS: i64 = 50; // 0.50% fee, expressed in basis points
static APP_NAME: &str = "MiniLedger";

// === 2. module + visibility + newtype (tuple struct) + operator overload ===
mod money {
    use std::fmt;
    use std::ops::Add;

    // A "newtype": a tuple struct wrapping one i64. Derives hand it free behavior.
    // Copy => cheap value semantics (assigning does NOT move it away).
    #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
    pub struct Cents(pub i64);

    // Operator overloading: implementing std::ops::Add lets us write `a + b`.
    impl Add for Cents {
        type Output = Cents; // associated type required by the Add trait
        fn add(self, rhs: Cents) -> Cents {
            Cents(self.0 + rhs.0)
        }
    }

    // Implementing Display controls how `{}` prints the type.
    impl fmt::Display for Cents {
        fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
            write!(f, "${} {:.02}", self.0 / 100, (self.0 % 100).abs())
        }
    }
}
use money::Cents;

// === 3. enum where each variant carries different data ===
#[derive(Debug, Clone)]
enum Txn {
    Deposit { account: String, cents: i64 },
    Withdraw { account: String, cents: i64 },
    Transfer { from: String, to: String, cents: i64 },
}

// === 4. custom error type + Display + Error + From (this is what powers `?`) ===
#[derive(Debug)]
enum LedgerError {
    InsufficientFunds { account: String, needed: i64, have: i64 },
    BadAmount(std::num::ParseIntError),
}

impl fmt::Display for LedgerError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {

```

```

        match self {
            LedgerError::InsufficientFunds { account, needed, have } => {
                write!(f, "{account}: need {needed}, have {have}")
            }
            LedgerError::BadAmount(e) => write!(f, "bad amount: {e}"),
        }
    }
}

impl Error for LedgerError {} // opt into the std error trait (uses Display + Debug)

// `` auto-converts a ParseIntError into our error through this From impl.
impl From<std::num::ParseIntError> for LedgerError {
    fn from(e: std::num::ParseIntError) -> Self {
        LedgerError::BadAmount(e)
    }
}

// A small fallible helper: `` on parse() triggers the From impl above.
fn parse_amount(s: &str) -> Result<i64, LedgerError> {
    let cents: i64 = s.trim().parse()?; // ParseIntError -> LedgerError, automatically
    Ok(cents)
}

// === 5. trait: supertrait bound + required method + default method (dyn-friendly) ===
trait Rail: fmt::Debug {
    // ``: fmt::Debug` is a supertrait requirement
    fn name(&self) -> &str; // required: every impl must supply it
    fn settle(&self, c: Cents) -> String {
        // default: you get it free, can override
        format!("{self} settled {c}", self.name())
    }
}

#[derive(Debug)]
struct Ach;
#[derive(Debug)]
struct Wire {
    priority: bool,
}

impl Rail for Ach {
    fn name(&self) -> &str {
        "ACH"
    }
}

impl Rail for Wire {
    fn name(&self) -> &str {
        "WIRE"
    }
    fn settle(&self, c: Cents) -> String {
        // overriding the default method
        let tag = if self.priority { "PRIORITY" } else { "std" };
        format!("{WIRE}/{tag} settled {c}")
    }
}

// === 6. a struct that holds a borrow needs an explicit lifetime ===
struct Report<'a> {
    title: &'a str, // borrowed: cannot outlive the &str it points at
    lines: Vec<String>, // owned
}

impl<'a> Report<'a> {
    fn print(&self) {
        println!("== {} ==", self.title);
        for line in &self.lines {

```

```

        println!("{}", {line});
    }
}

// === 7. the core data: a struct with methods, &self vs &mut self ===
#[derive(Debug, Default)]
struct Ledger {
    balances: HashMap<String, i64>, // owned String keys -> i64 cents
}

impl Ledger {
    fn new() -> Self {
        Ledger { ..Default::default() } // struct-update syntax + Default
    }

    fn balance(&self, who: &str) -> i64 {
        // get() returns Option<i64>; copied() -> Option<i64>; unwrap_or supplies a default
        self.balances.get(who).copied().unwrap_or(0)
    }

    // &mut self: an exclusive borrow. Result makes the op fallible.
    fn apply(&mut self, txn: &Txn) -> Result<(), LedgerError> {
        match txn {
            Txn::Deposit { account, cents } => {
                *self.balances.entry(account.clone()).or_insert(0) += cents;
            }
            Txn::Withdraw { account, cents } => {
                let have = self.balance(account);
                if have < *cents {
                    return Err(LedgerError::InsufficientFunds {
                        account: account.clone(),
                        needed: *cents,
                        have,
                    });
                }
                *self.balances.get_mut(account).unwrap() -= cents;
            }
            Txn::Transfer { from, to, cents } => {
                // reuse apply() + '?' to bubble an error up out of the transfer
                self.apply(&Txn::Withdraw { account: from.clone(), cents: *cents })?;
                self.apply(&Txn::Deposit { account: to.clone(), cents: *cents })?;
            }
        }
        Ok(())
    }

    // Explicit lifetime: the returned &str borrows from `self`'s map keys.
    // (Elision would let you omit 'a here; written out so you can see it.)
    fn richest<'a>(&'a self) -> Option<(&'a str, i64)> {
        self.balances
            .iter()
            .max_by_key(|&(_, &bal)| bal)
            .map(|(name, &bal)| (name.as_str(), bal))
    }
}

// === 8. custom iterator: associated type `Item`, required `next`, free combinators ===
struct Countdown {
    n: u32,
}

impl Iterator for Countdown {
    type Item = u32; // the associated type the Iterator trait demands
    fn next(&mut self) -> Option<u32> {
        if self.n == 0 {

```

```

        None
    } else {
        self.n -= 1;
        Some(self.n + 1)
    }
}

// === 9. generics + trait bound + where-clause + impl-Trait return ===
fn describe_all<R>(rails: &[R]) -> Vec<String>
where
    R: Rail, // bound: R must implement Rail (static dispatch / monomorphized)
{
    rails.iter().map(|r| r.settle(Cents(100))).collect()
}

fn fee_schedule() -> impl Iterator<Item = i64> {
    // returns an opaque iterator type
    (1_i64..=3).map(|tier| tier * FEE_BPS)
}

// === 10. closures: an `Fn` taken as a generic parameter ===
fn apply_fee<F: Fn(i64) -> i64>(amount: i64, f: F) -> i64 {
    f(amount)
}

fn main() -> Result<(), Box<dyn Error>> {
    // main returns Result so `?` works here too

    // --- variable flavors ---
    let immutable = 42; // inferred i32, immutable by default
    let mut counter: u32 = 0; // explicit type + mutable
    let pi: f64 = 3.14159; // float
    let grade: char = 'A'; // char = one Unicode scalar (4 bytes)
    let active: bool = true; // bool
    let owned: String = String::from("syndication"); // owned, heap-allocated
    let view: &str = &owned[0..4]; // borrowed slice into it -> "synd"
    let trio: (u8, &str, f64) = (1, "x", 0.5); // tuple
    let arr: [i64; 3] = [10, 20, 30]; // fixed-size array
    let slice: &[i64] = &arr[1..]; // slice view -> [20, 30]
    let shadowed = immutable; // shadowing:
    let shadowed = shadowed.to_string(); // same name, different type (i32 -> String)

    println!("{APP_NAME}: {immutable} {pi} {grade} {active} {view} {trio:?} {slice:?} {shadowed}");

    // --- Result + `?` + From conversion ---
    let amt = parse_amount(" 4200 ")?; // returns LedgerError, lifted to Box<dyn Error> by main
    println!("parsed amount = {}", Cents(amt));

    // --- newtype + operator overload + Copy ---
    let a = Cents(150);
    let b = Cents(250);
    let total = a + b; // uses our Add impl; a and b stay usable because Cents is Copy
    println!("fee math: {a} + {b} = {total}");

    // --- closures: Fn passed in, FnMut capturing &mut ---
    let with_fee = apply_fee(10_000, |x| x + x * FEE_BPS / 10_000); // Fn closure
    let mut bump = || counter += 1; // FnMut: captures counter by &mut
    bump();
    bump();
    println!("with_fee={with_fee}, counter={counter}");

    // --- custom iterator + standard combinators ---
    let countdown: Vec<u32> = Countdown { n: 3 }.collect();
    let squared_sum: u32 = Countdown { n: 3 }.map(|x| x * x).sum();

```

```

println!("countdown={countdown:?}", squared_sum={squared_sum});
println!("fees={:?}", fee_schedule().collect::<Vec<_>>());

// --- trait objects: dynamic dispatch through Box<dyn Rail> ---
let rails: Vec<Box<dyn Rail>> = vec![Box::new(Ach), Box::new(Wire { priority: true })];
for r in &rails {
    println!("{}", r.settle(Cents(999))); // resolved at runtime via a vtable
}
// static-dispatch counterpart (one specialized copy generated at compile time):
println!("{}", describe_all(&[Ach, Ach]));

// --- concurrency: Arc<Mutex<_>> shared state + mpsc channel for receipts ---
let ledger = Arc::new(Mutex::new(Ledger::new()));
let batches: Vec<Vec<Txn>> = vec![
    vec![
        Txn::Deposit { account: "merchant".into(), cents: 100_000 },
        Txn::Withdraw { account: "merchant".into(), cents: 30_000 },
    ],
    vec![
        Txn::Deposit { account: "alice".into(), cents: 50_000 },
        Txn::Transfer { from: "alice".into(), to: "bob".into(), cents: 20_000 },
        Txn::Withdraw { account: "bob".into(), cents: 999_999 }, // will fail -> Err path
    ],
];

let (tx, rx) = mpsc::channel::<String>();
let mut handles = Vec::new();

for (i, batch) in batches.into_iter().enumerate() {
    let ledger = Arc::clone(&ledger); // bump the atomic refcount, move the clone in
    let tx = tx.clone(); // each worker gets its own sender
    let handle = thread::spawn(move || {
        // `move` hands ownership of captures to the thread
        for txn in &batch {
            let mut guard = ledger.lock().unwrap(); // acquire the Mutex
            match guard.apply(txn) {
                Ok(()) => {
                    let _ = tx.send(format!("worker {i}: ok    {txn:?}"));
                }
                Err(e) => {
                    let _ = tx.send(format!("worker {i}: ERR  {e}"));
                }
            }
        }
        // `guard` dropped each loop -> Mutex released
    });
    handles.push(handle);
}
drop(tx); // drop the original sender so the channel can close

for h in handles {
    h.join().unwrap(); // wait for every worker to finish
}

// `while let` drains the channel until all senders are gone
let mut receipts = Vec::new();
while let Ok(msg) = rx.recv() {
    receipts.push(msg);
}
receipts.sort();

// --- pattern matching: destructuring, ranges, guards, `@` binding ---
let final_ledger = ledger.lock().unwrap();
if let Some((name, bal)) = final_ledger.richest() {
    let tier = match bal {
        0 => "empty".to_string(),

```

```

        n @ 1..=49_999 => format!("small({n})"), // `@` binds the matched value
        50_000..=99_999 => "mid".to_string(),   // inclusive range pattern
        _ => "whale".to_string(),               // catch-all
    };
    println!("richest: {name} = {} ({tier})", Cents(bal));
}

// --- build a Report that borrows a &str (the lifetime from section 6 in action) ---
let report = Report { title: APP_NAME, lines: receipts };
report.print();

Ok(())
}

// === 11. unit test (run with: rustc --test miniledger.rs && ./miniledger) ===
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn deposit_then_withdraw() {
        let mut l = Ledger::new();
        l.apply(&Txn::Deposit { account: "a".into(), cents: 100 }).unwrap();
        l.apply(&Txn::Withdraw { account: "a".into(), cents: 40 }).unwrap();
        assert_eq!(l.balance("a"), 60);
    }
}

```